

Documentação de Projeto

para o sistema

OficinaPro

Sistema de Gestão para Oficinas Mecânicas

Versão 1.0

Projeto de sistema elaborado pela aluna **Bruna Lopes de Souza**
como parte da disciplina **Projeto de Software**.

Prof. Dr. João Paulo Aramuni

Pontifícia Universidade Católica de Minas Gerais (PUC Minas)

Belo Horizonte, junho de 2026

Tabela de Conteúdo

1. Introdução4

2. Modelos de Usuário e Requisitos4

2.1 Descrição de Atores4

2.2 Modelo de Casos de Uso e Histórias de Usuários5

2.3 Diagrama de Sequência do Sistema e Contrato de Operações6

3. Modelos de Projeto9

3.1 Arquitetura9

3.2 Diagrama de Componentes e Implantação10

3.3 Diagrama de Classes11

3.4 Diagramas de Sequência12

3.5 Diagramas de Comunicação13

3.6 Diagramas de Estados14

4. Modelos de Dados16

Histórico de Revisões

Nome	Data	Razões para Mudança	Versão
Bruna Lopes de Souza	07/06/2026	Criação inicial do documento de projeto.	1.0

1. Introdução

Este documento reúne os modelos de domínio e os modelos de projeto do **OficinaPro**, um sistema de gestão para oficinas mecânicas automotivas de pequeno e médio porte. As Seções 2 e 3 consolidam a visão de domínio que serve de base para a descrição do problema, dos requisitos e das decisões de projeto.

A ideia do sistema é informatizar a operação da oficina e acabar com os controles feitos em papel e em planilhas soltas. Toda a rotina de atendimento passa a ficar em um só lugar: do agendamento e da abertura da Ordem de Serviço (OS) até a elaboração do orçamento, a execução, o controle de estoque de peças e o faturamento. Assim, cada cliente e cada veículo ganham um histórico completo e fácil de consultar.

Na prática, é comum a oficina perder o histórico de manutenção, ter dificuldade para montar orçamentos consistentes e não saber ao certo quais peças foram usadas em cada serviço. O cliente também costuma ficar sem informação sobre o andamento do reparo. O OficinaPro ataca esses pontos com um fluxo padronizado de OS, um orçamento que o próprio cliente aprova e a baixa automática do estoque.

O escopo cobre o cadastro de clientes e veículos, o agendamento, a OS, o orçamento, a execução, o estoque e o pagamento. Ficam de fora folha de pagamento, contabilidade fiscal e emissão de nota fiscal eletrônica; o sistema apenas gera um comprovante interno de pagamento. Vale lembrar que, conforme a proposta do trabalho, este documento trata do projeto, da diagramação e da arquitetura da aplicação, sem implementação de código.

2. Modelos de Usuário e Requisitos

2.1 Descrição de Atores

Nesta subseção são apresentados os atores que interagem com o sistema, sejam pessoas (atores humanos) ou sistemas externos (atores de sistema).

Ator	Descrição
Cliente	Proprietário do veículo. Solicita agendamentos, acompanha o andamento da OS, aprova ou recusa orçamentos e efetua o pagamento.
Atendente (Consultor de Serviços)	Responsável pela recepção. Cadastra clientes e veículos, abre ordens de serviço, elabora orçamentos e registra pagamentos.
Mecânico	Executa o diagnóstico e os serviços. Registra peças e horas trabalhadas e atualiza o status da OS.
Gerente	Administra o catálogo de serviços, o controle de estoque e os usuários, além de gerar relatórios gerenciais.
Gateway de Pagamento (sistema)	Ator de sistema externo responsável por processar as transações de pagamento (cartão de crédito/débito e PIX).
Fornecedor (sistema)	Ator de sistema externo que recebe as solicitações de reposição quando uma peça atinge o estoque mínimo.

2.2 Modelo de Casos de Uso e Histórias de Usuários

A Figura 1 apresenta o diagrama de casos de uso do sistema. Cada caso de uso possui um identificador (UC-01, UC-02, ...) que serve de referência no restante do documento.

Diagrama de Casos de Uso - OficinaPro

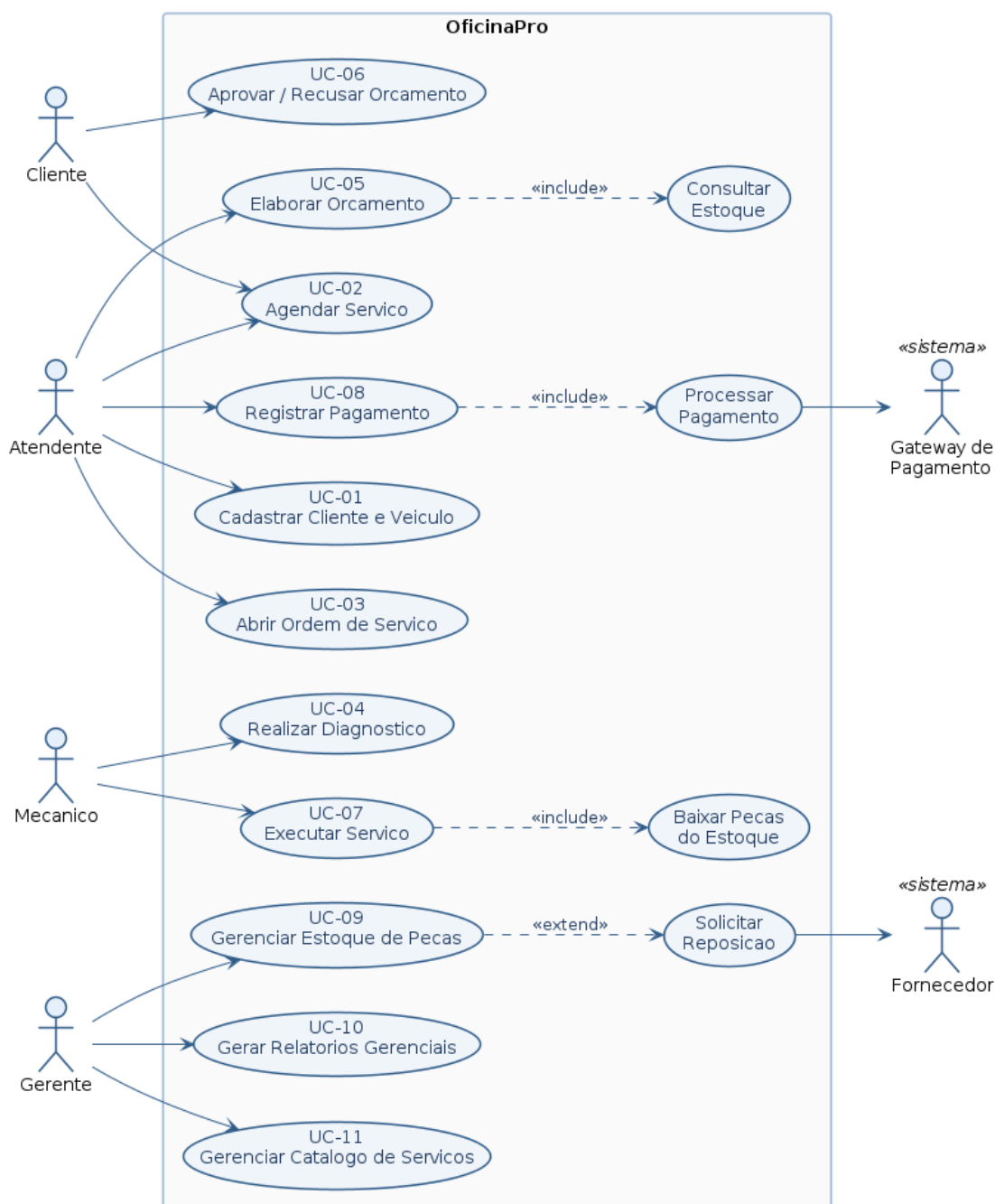


Figura 1 – Diagrama de casos de uso do OficinaPro

ID	Caso de uso	Ator principal
UC-01	Cadastrar Cliente e Veículo	Atendente
UC-02	Agendar Serviço	Cliente / Atendente
UC-03	Abrir Ordem de Serviço	Atendente
UC-04	Realizar Diagnóstico	Mecânico

UC-05	Elaborar Orçamento	Atendente
UC-06	Aprovar / Recusar Orçamento	Cliente
UC-07	Executar Serviço	Mecânico
UC-08	Registrar Pagamento	Atendente
UC-09	Gerenciar Estoque de Peças	Gerente
UC-10	Gerar Relatórios Gerenciais	Gerente
UC-11	Gerenciar Catálogo de Serviços	Gerente

A partir desses casos de uso, foram escritas algumas histórias de usuário no formato “Como < papel>, quero <objetivo>, para <benefício>”:

- **HU-01:** Como cliente, quero acompanhar o status da minha OS pelo aplicativo, para saber quando o veículo estará pronto.
- **HU-02:** Como atendente, quero elaborar um orçamento com serviços e peças, para enviá-lo ao cliente antes de iniciar a execução.
- **HU-03:** Como cliente, quero aprovar ou recusar o orçamento remotamente, para autorizar o reparo sem precisar ir à oficina.
- **HU-04:** Como gerente, quero ser alertado quando uma peça atingir o estoque mínimo, para solicitar a reposição ao fornecedor.

2.3 Diagrama de Sequência do Sistema e Contrato de Operações

Esta subseção traz os diagramas de sequência do sistema (SSD) de três casos de uso: UC-03 (Abrir OS), UC-05/UC-06 (Elaborar e Aprovar Orçamento) e UC-08 (Registrar Pagamento). Em seguida, vêm os contratos das principais operações de sistema que apareceram nesses diagramas.

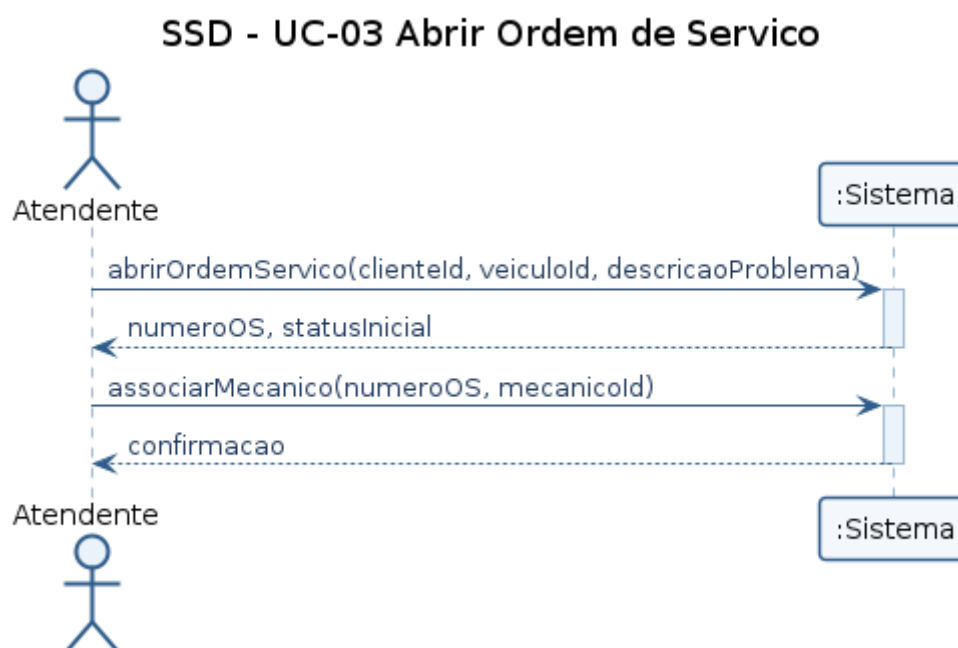


Figura 2 – SSD do caso de uso UC-03 – Abrir Ordem de Serviço

SSD - UC-05/06 Elaborar e Aprovar Orçamento

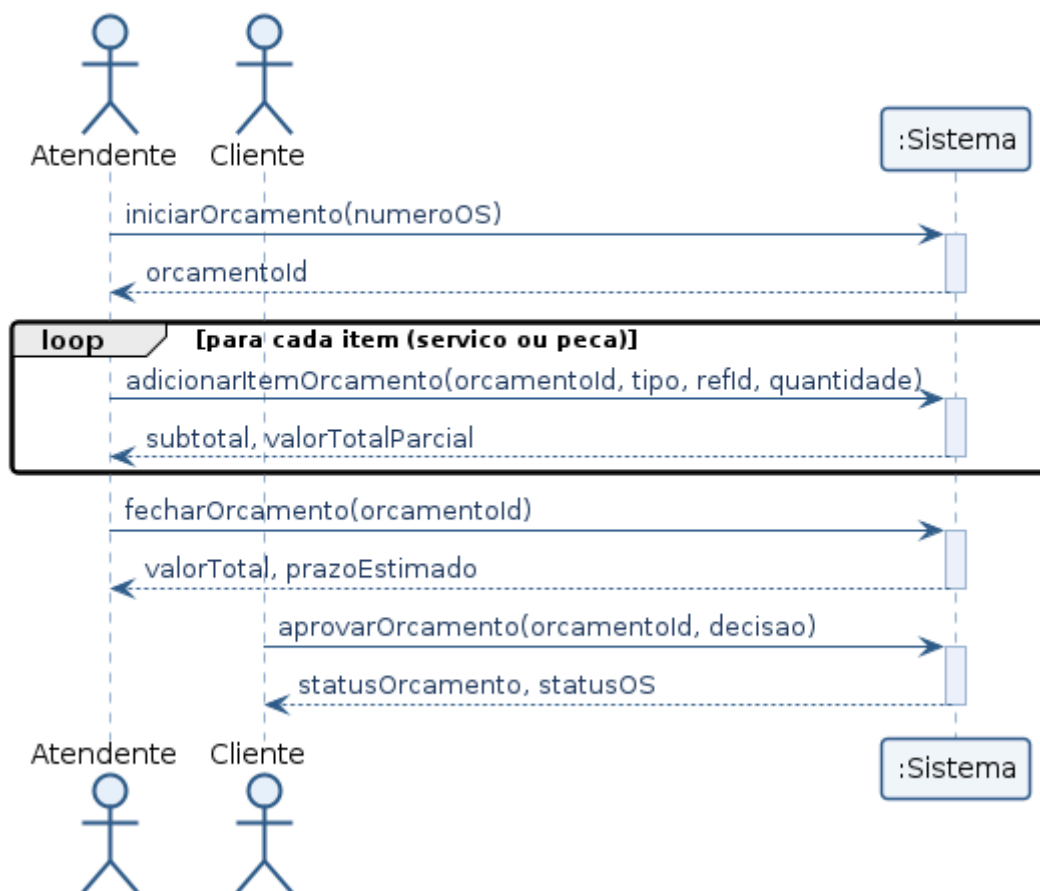


Figura 3 – SSD dos casos de uso UC-05/UC-06 – Elaborar e Aprovar Orçamento

SSD - UC-08 Registrar Pagamento

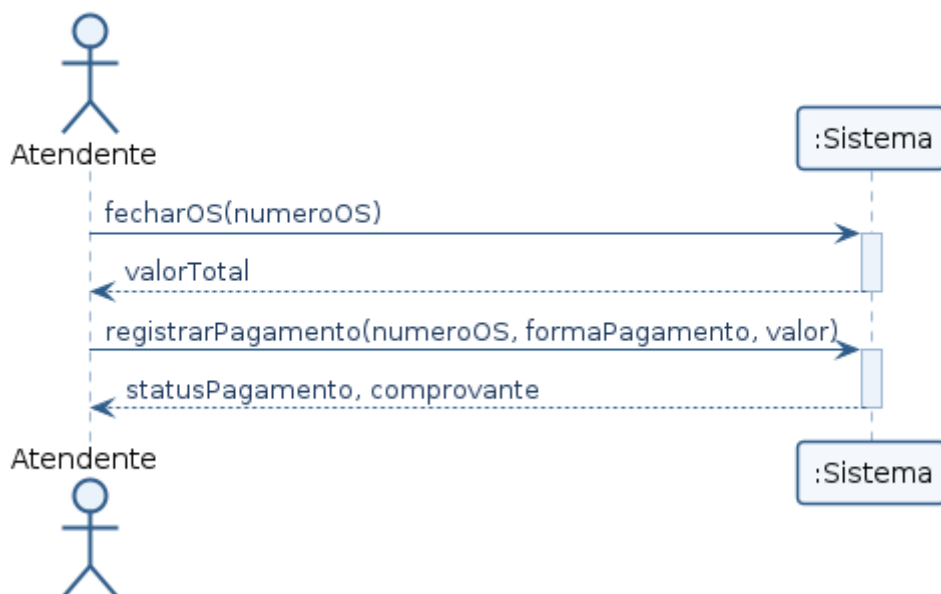


Figura 4 – SSD do caso de uso UC-08 – Registrar Pagamento

Os contratos a seguir descrevem cada operação disparada pelos atores nos SSDs, com suas pré e pós-condições.

Contrato CO-01

Operação	abrirOrdemServico(clienteld, veiculold, descricaoProblema)
Referências cruzadas	UC-03 (Abrir Ordem de Serviço)
Pré-condições	O cliente e o veículo já estão cadastrados no sistema. O veículo não possui outra OS em aberto.
Pós-condições	Uma nova instância de OrdemServico foi criada e persistida. O status da OS foi definido como ABERTA. A OS foi associada ao veículo e ao atendente responsável.

Contrato CO-02

Operação	adicionarItemOrcamento(orçamentold, tipo, refld, quantidade)
Referências cruzadas	UC-05 (Elaborar Orçamento)
Pré-condições	Existe um orçamento em elaboração para a OS. O serviço (refld) ou a peça (refld) está cadastrado; havendo peça, há quantidade suficiente em estoque.
Pós-condições	Um ItemServico ou ItemPeca foi associado ao orçamento. O valor total do orçamento foi recalculado.

Contrato CO-03

Operação	aprovarOrcamento(orçamentold, decisao)
Referências cruzadas	UC-06 (Aprovar / Recusar Orçamento)
Pré-condições	O orçamento está no status ENVIADO.
Pós-condições	Se aprovado: o orçamento passou a APROVADO e a OS passou a EM_EXECUCAO. Se recusado: o orçamento passou a RECUSADO e a OS passou a CANCELADA.

Contrato CO-04

Operação	registrarPagamento(numeroOS, formaPagamento, valor)
Referências cruzadas	UC-08 (Registrar Pagamento)
Pré-condições	A OS está no status CONCLUIDA. O valor informado corresponde ao valor total da OS.

Pós-condições

A transação foi processada pelo gateway e aprovada.
 Uma instância de Pagamento foi persistida e vinculada à OS.
 O status da OS passou a FATURADA.

3. Modelos de Projeto

3.1 Arquitetura

A arquitetura do OficinaPro é organizada em camadas, com uma separação clara de responsabilidades inspirada na **Clean Architecture** e em uma leitura próxima à do **C4 Model**. A camada de domínio não depende de framework, e o acesso a recursos externos (banco de dados, gateway de pagamento e e-mail) é feito por adaptadores. Essa organização facilita os testes e ajuda a manter o acoplamento baixo.

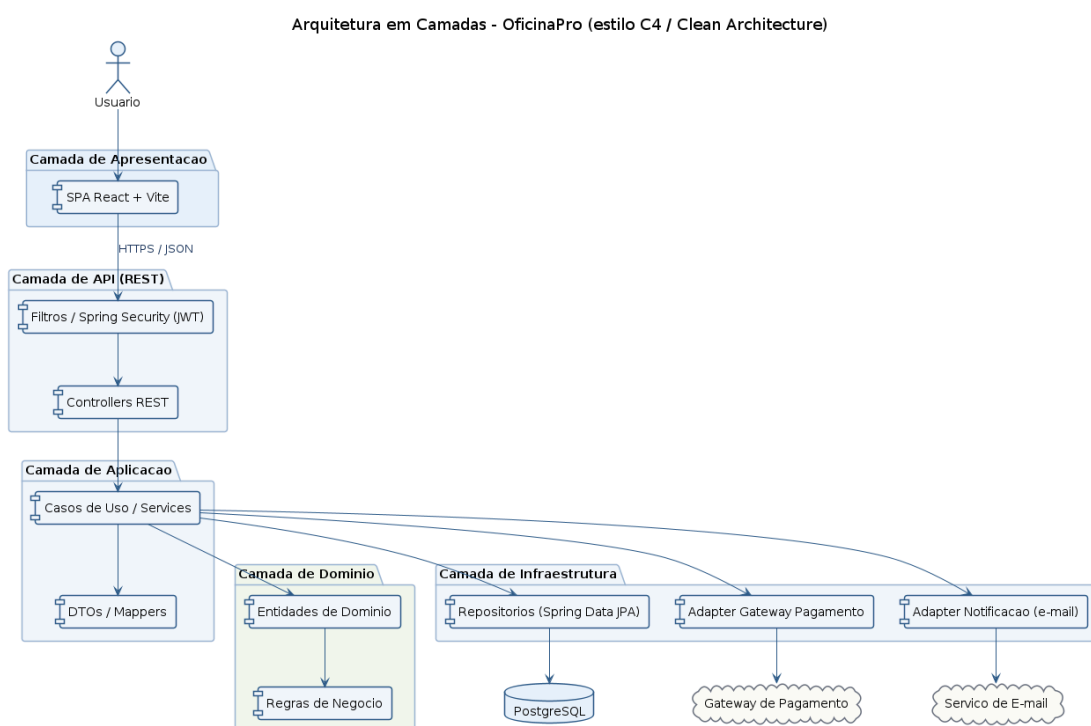


Figura 5 – Visão arquitetural em camadas do OficinaPro

Cada camada tem um papel bem definido:

- Apresentação (SPA em React + Vite): a interface usada por atendentes, mecânicos e gerentes, que conversa com o back-end por HTTPS/JSON.
- API REST (Controllers + Spring Security): expõe os endpoints, valida a entrada e cuida da autenticação e autorização com JWT.
- Aplicação (Services / Casos de Uso): coordena as regras de negócio, aciona os repositórios e os adaptadores e converte as entidades em DTOs.
- Domínio (Entidades e Regras de Negócio): o núcleo do sistema, com as entidades OrdemServico, Orcamento, Estoque e suas invariantes.
- Infraestrutura: os repositórios Spring Data JPA e os adaptadores para o gateway de pagamento e o serviço de e-mail.

Entre os padrões de projeto, o sistema usa Repository para abstrair a persistência, Service Layer para concentrar as regras de negócio, DTO para transportar dados entre as camadas e Adapter para conversar com os sistemas externos. Essa divisão tem um custo: surge mais código de mapeamento entre entidade e DTO e um número maior de classes. Em compensação, o acoplamento cai e fica mais simples trocar uma tecnologia de infraestrutura sem mexer no domínio.

3.2 Diagrama de Componentes e Implantação

O diagrama de componentes (Figura 6) mostra os módulos lógicos do sistema e suas dependências; o de implantação (Figura 7) indica onde cada parte roda.

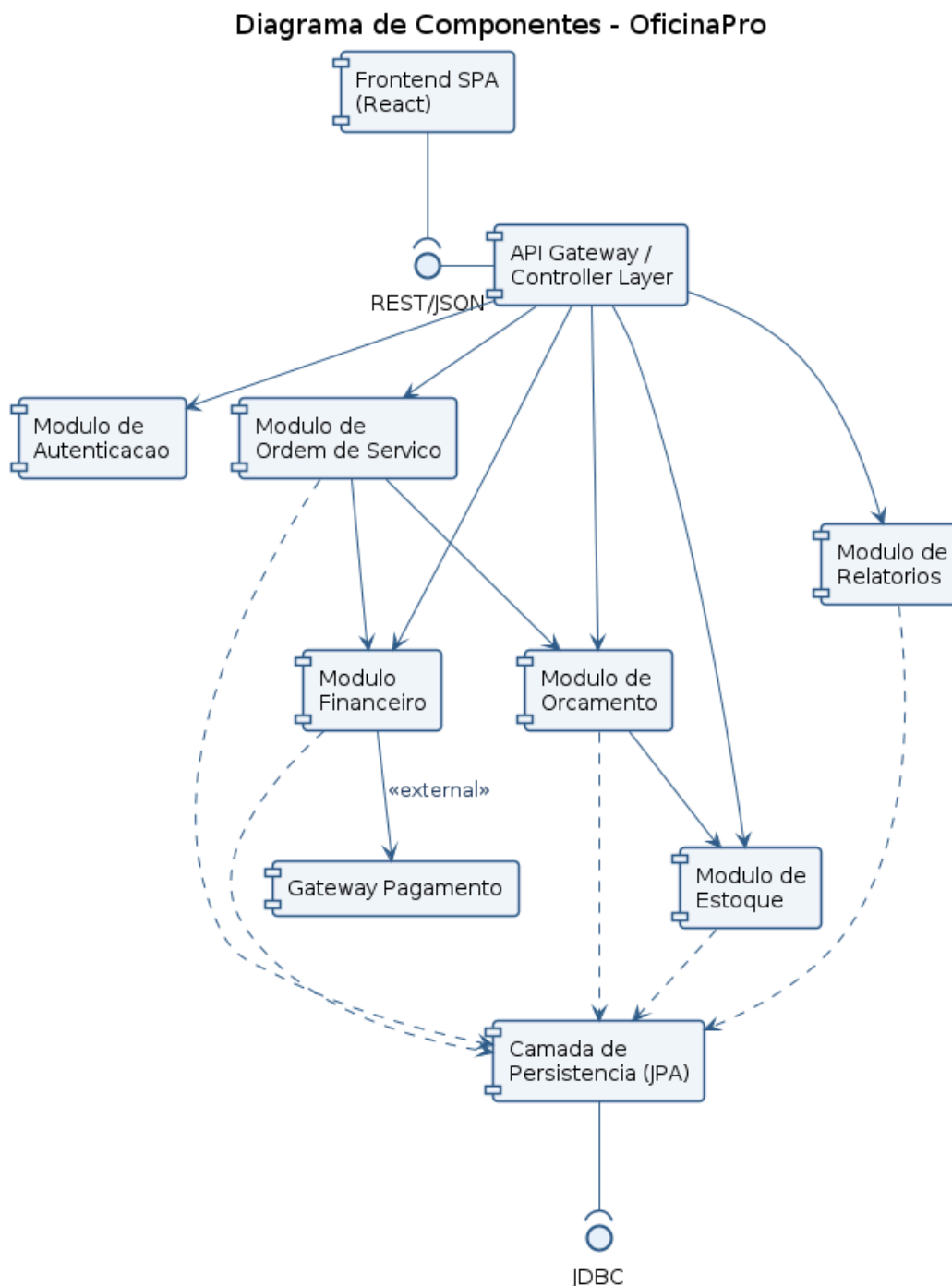


Figura 6 – Diagrama de componentes do OficinaPro

O front-end consome a API pela interface REST/JSON. A camada de controle repassa as chamadas aos módulos de negócio (Autenticação, Ordem de Serviço, Orçamento, Estoque, Financeiro e Relatórios), que por sua vez usam a camada de persistência (JPA) sobre o banco. O módulo Financeiro é o que se integra ao gateway de pagamento externo.

Diagrama de Implantação - OficinaPro

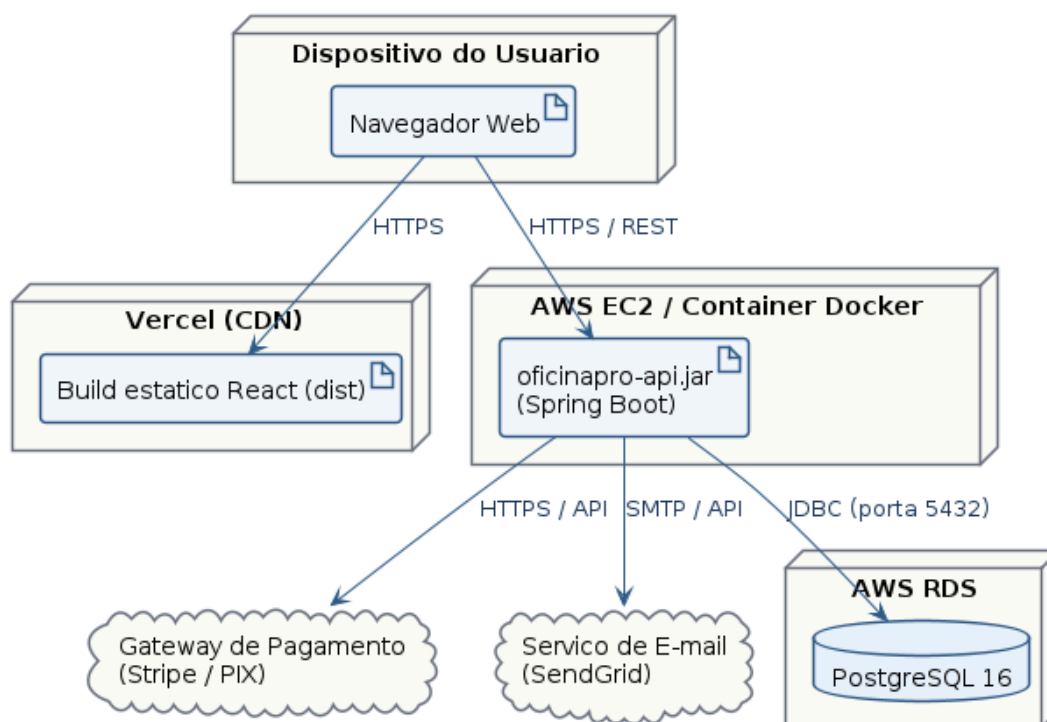


Figura 7 – Diagrama de implantação do OficinaPro

Na implantação, o build estático do React é servido por uma CDN (Vercel), a API Spring Boot roda em um contêiner Docker numa instância AWS EC2 e o banco PostgreSQL fica no AWS RDS. As integrações externas (gateway de pagamento e serviço de e-mail) são acessadas por HTTPS.

3.3 Diagrama de Classes

A Figura 8 traz o diagrama de classes de projeto, com atributos, operações, relacionamentos e multiplicidades.

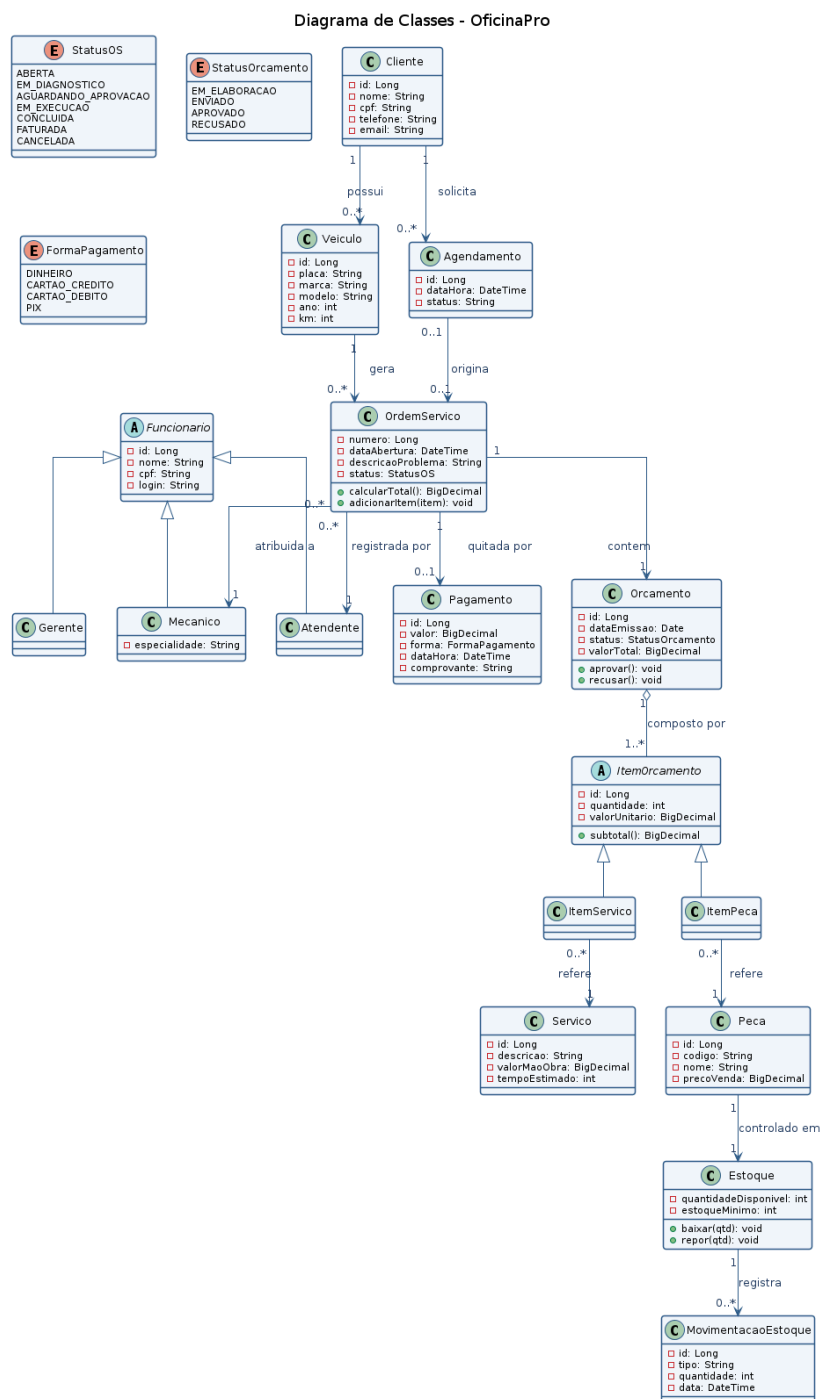


Figura 8 – Diagrama de classes do OficinaPro

Há duas hierarquias de herança no modelo. A de Funcionario se especializa em Atendente, Mecanico e Gerente, e a de ItemOrcamento se divide em ItemServico e ItemPeca. A classe OrdemServico é o centro do diagrama: ela se relaciona com Veiculo, Mecanico, Atendente, Orcamento e Pagamento. As enumerações StatusOS, StatusOrcamento e FormaPagamento representam os estados e as categorias de negócio. Por fim, a composição entre Orcamento e ItemOrcamento indica que os itens não fazem sentido sem o orçamento que os contém.

3.4 Diagramas de Sequência

Os diagramas a seguir realizam casos de uso em nível de projeto e mostram como controllers, services e repositórios colaboram entre si.

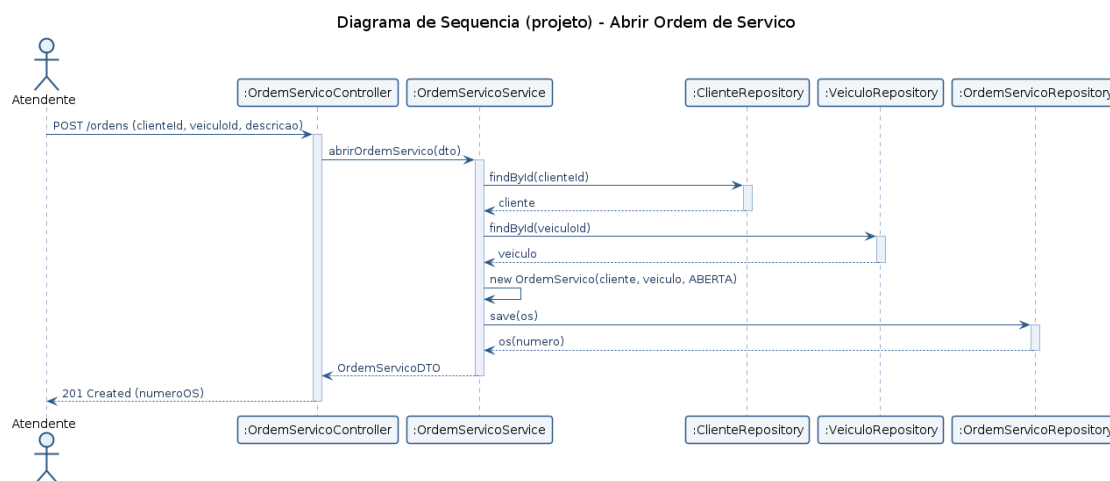


Figura 9 – Diagrama de sequência – Abrir Ordem de Serviço (UC-03)

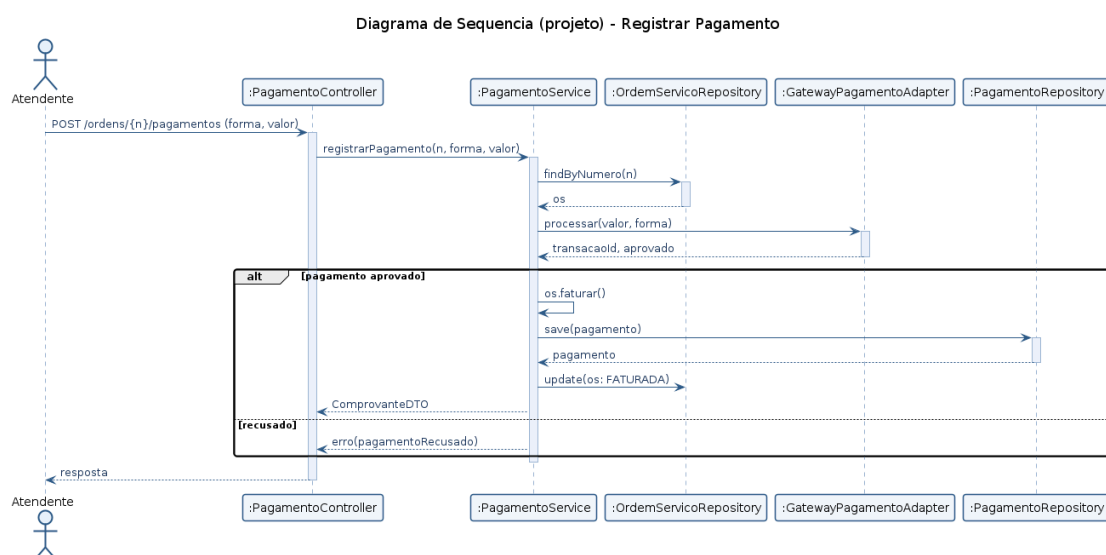
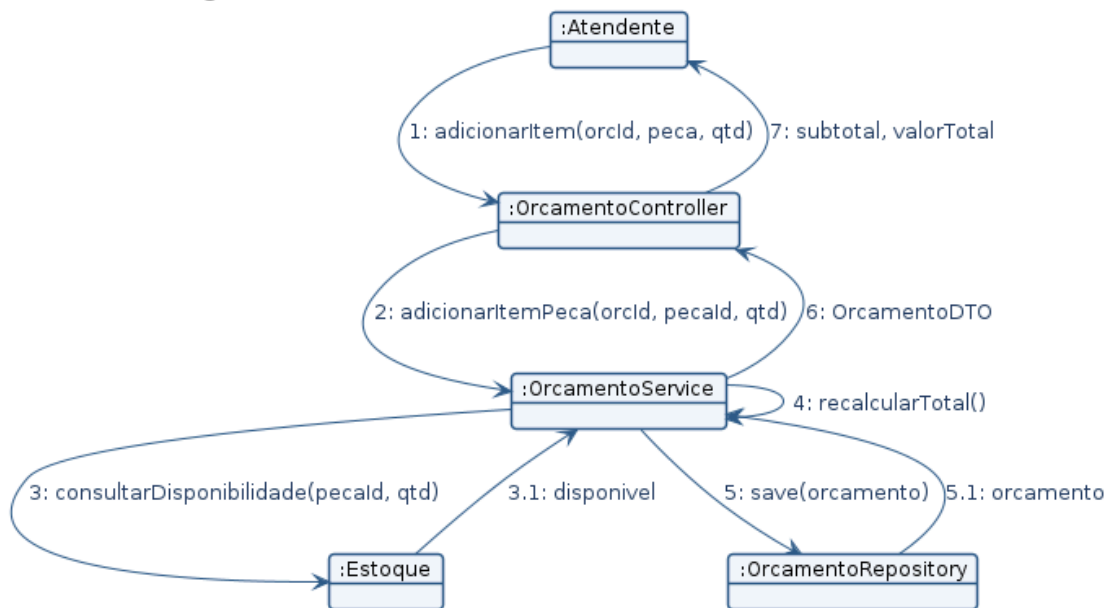


Figura 10 – Diagrama de sequência – Registrar Pagamento (UC-08)

No fluxo de pagamento, o fragmento alt separa o que acontece quando a transação é aprovada do que acontece quando ela é recusada pelo gateway. O pagamento só é gravado e o status da OS só muda para FATURADA no caminho de sucesso.

3.5 Diagramas de Comunicação

A Figura 11 mostra o mesmo cenário de elaboração de orçamento, agora pela ótica da comunicação, com as mensagens numeradas para deixar clara a ordem em que são trocadas entre os objetos.

Diagrama de Comunicacao - Elaborar Orcamento (UC-05)*Figura 11 – Diagrama de comunicação – Elaborar Orçamento (UC-05)***3.6 Diagramas de Estados**

Os diagramas de estados modelam o ciclo de vida das entidades de negócio mais dinâmicas: a Ordem de Serviço (Figura 12) e o Orçamento (Figura 13).

Diagrama de Estados - Ordem de Serviço

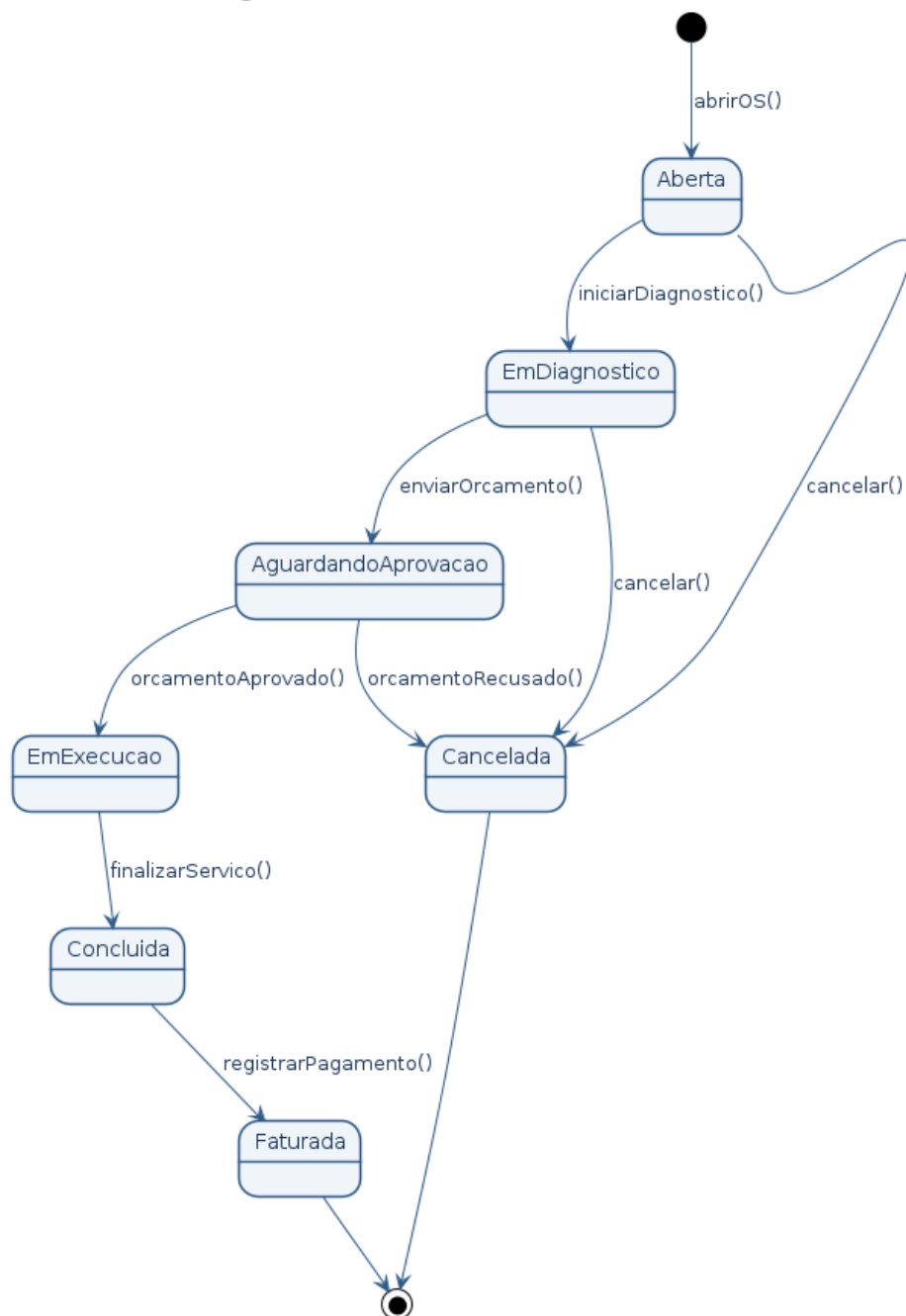


Figura 12 – Diagrama de estados da Ordem de Serviço

Diagrama de Estados - Orcamento

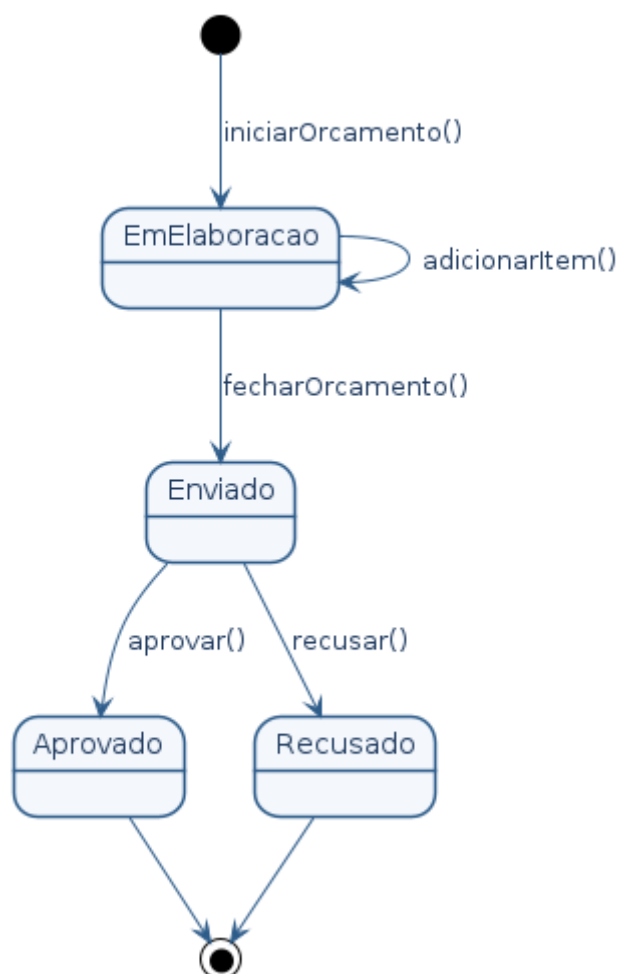


Figura 13 – Diagrama de estados do Orçamento

4. Modelos de Dados

Esta seção apresenta o esquema de banco de dados relacional e a estratégia de mapeamento entre as representações de objetos (classes de domínio) e não-objetos (tabelas relacionais).

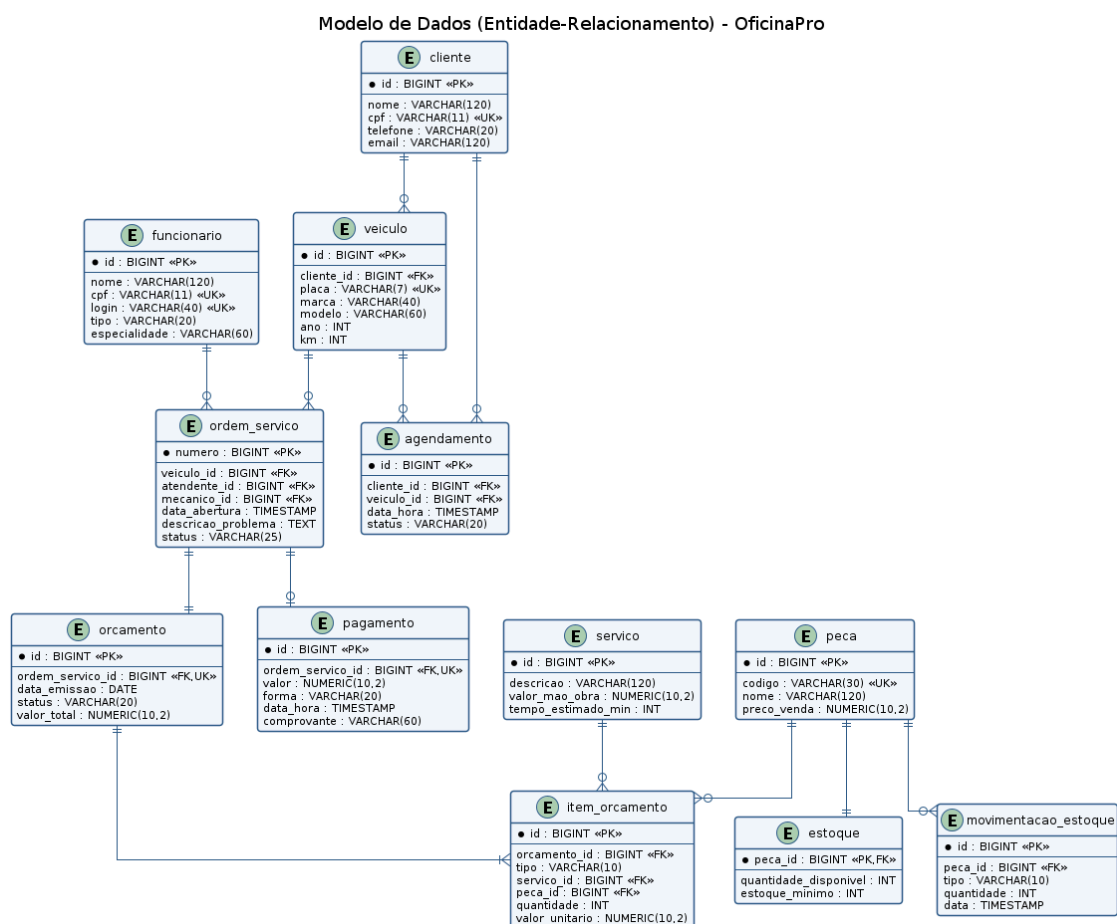


Figura 14 – Modelo de dados (entidade-relacionamento) do OficinaPro

O modelo está normalizado até a 3FN e usa chaves substitutas (BIGINT autoincremental) como chave primária. As restrições de unicidade ficam em cpf, placa e código de peça. Os relacionamentos um-para-muitos são feitos por chaves estrangeiras, e o relacionamento 1:1 entre ordem_servico e orcamento (assim como com pagamento) é garantido por uma FK com restrição UNIQUE.

Sobre o mapeamento entre objetos e tabelas (JPA/Hibernate), valem alguns pontos:

- A hierarquia Funcionario foi mapeada com SINGLE_TABLE: uma única tabela funcionario com a coluna tipo para diferenciar Atendente, Mecanico e Gerente. A escolha prioriza o desempenho das consultas em vez da normalização.
- ItemOrcamento segue a mesma ideia (tabela item_orcamento com a coluna tipo e FKs opcionais para servico e peca, conforme o subtipo).
- As enumerações (StatusOS, StatusOrcamento, FormaPagamento) são gravadas como texto, com @Enumerated(EnumType.STRING), o que evita problemas caso a ordem das constantes mude.
- Os relacionamentos 1:N (como cliente para veiculo) usam @OneToMany/@ManyToOne; a composição de Orcamento com ItemOrcamento usa cascade ALL e orphanRemoval, já que os itens dependem do orçamento.
- Entre peca e estoque há um 1:1, com a peça como dona da relação; a tabela movimentacao_estoque guarda o histórico de entradas e saídas para fins de auditoria.

Por fim, a baixa de peças (UC-07) e a mudança de status da OS acontecem na mesma transação. Isso garante atomicidade: ou o serviço é registrado e as peças saem do estoque, ou, em caso de falha, nada é alterado.